CSE 370: Database Systems
Project Report
Project Title: Splitflix

Group No :03, CSE370 Lab Section :05, Spring 2026		
ID	**Name**	**Contribution**
24101562	Shadman Shakib	Authentication, User/owner panel specification, Owner dashboard, waitlist queue, leave queue
24101175	Tanvir Muhtady	Notification broadcast, chatroom, recommendation
23201017	Tanvir Ahmed	User dashboard, Review System, google form payment system

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	4
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7-8
6	Frontend Development	9-11
7	Backend Development	10-17
8	Source Code Repository	17
9	Conclusion	17
10	References	17

Introduction

[Briefly describe the idea of your project]

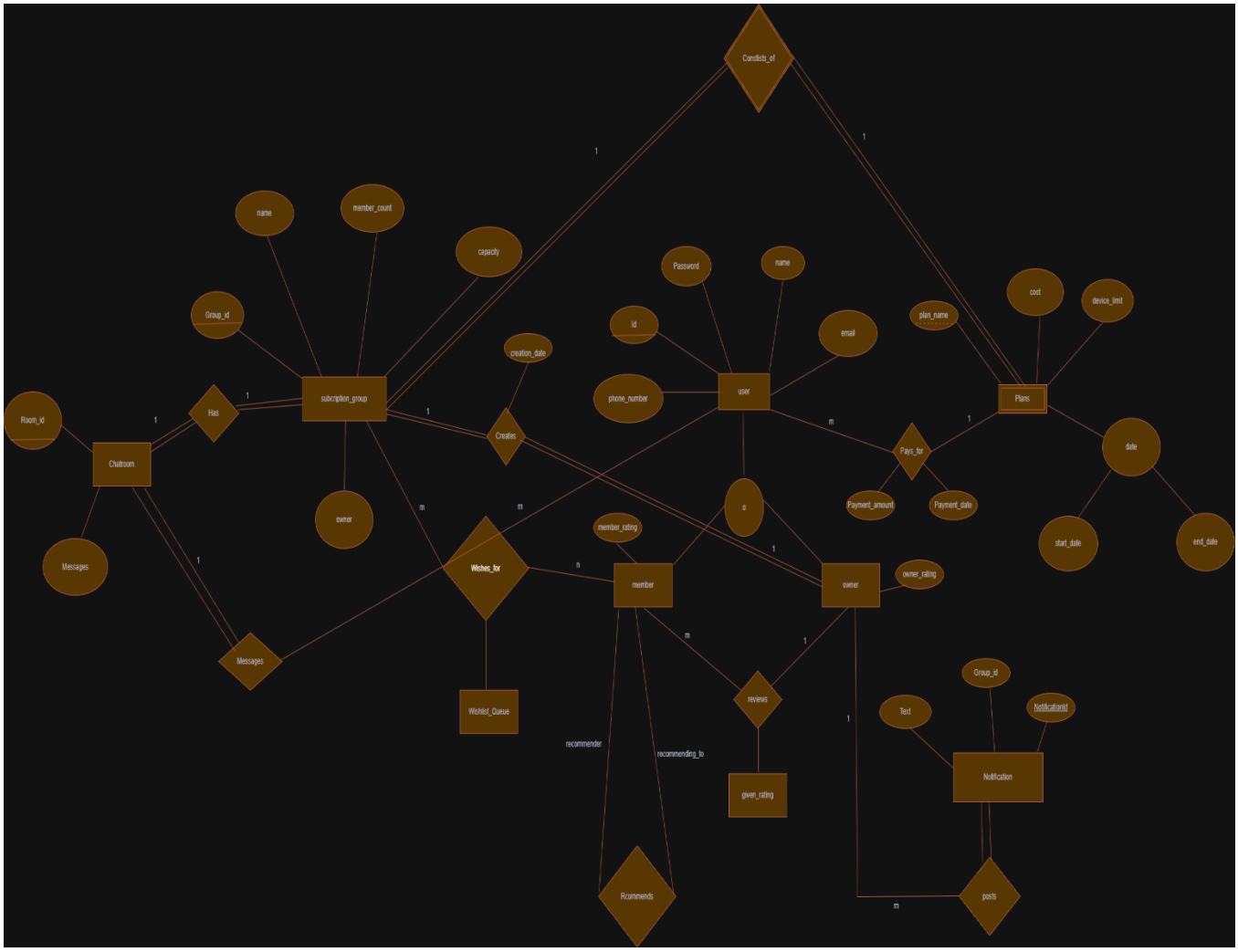
Our project Splitflix, gives the user the choice to share digital subscription plan for five popular platform worldwide- Netflix, Amazon Prime, Disney Hotstar, Spotify and HBO max. User can join available groups according to their own convenience. Top features of the group are:

1. User/Owner Specification
2. Owner verification
3. Payment System via google form
4. Three specialized groups (partially filled, filled, open group as owner)
5. Waitlist queue (Waiting members can join in a group up to 7 days of the upcoming billing cycle)
6. Leave queue (Members wanting to leave can leave the group in the next billing cycle)
7. Owner member review System
8. User dashboard to browse to view groups
9. Recommendation system among members
10. Common chatroom
11. Broadcasting system among members via owner

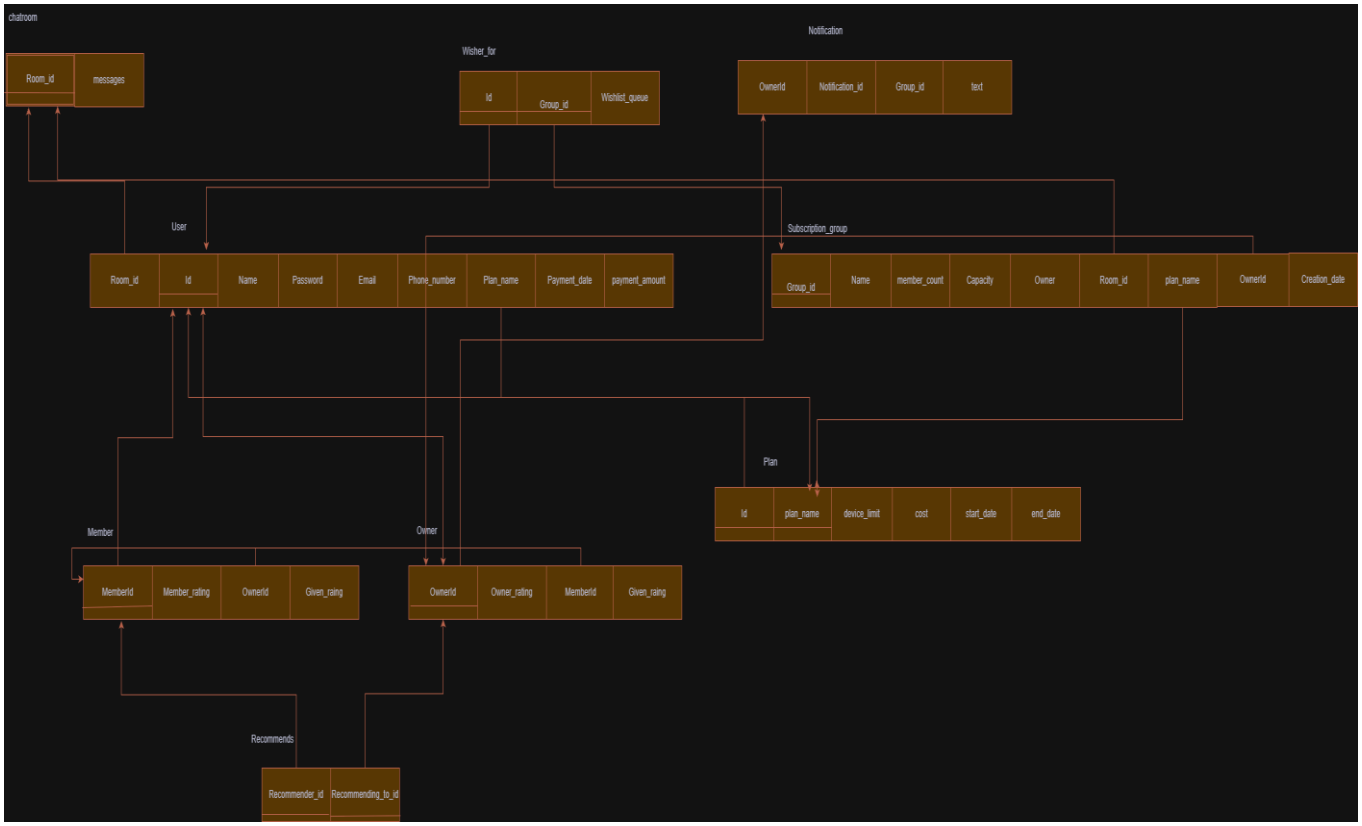
Project Features

ID, Name	Features [3 per member]	
24101562, Shadman Shakib	Ft 1	User/ Owner Specification
	Ft 2	Owner Dashboard, group creation for owner, Payment status reset to uncleared at the start of each billing cycle
	Ft 3	Waitlist queue and leave queue
24101175, Tanvir Muhtady	Ft 1	Notification broadcast via owner to the member
	Ft 2	Recommendation system among members
	Ft 3	Community chatroom
23201017, Tanvir Ahmed	Ft 1	User Owner review System
	Ft 2	User dashboard
	Ft 3	Payment via google form

ER/EER Diagram



Schema Diagram



Normalization

a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.

Yes, the converted relational schema is already in First Normal Form (1NF).

Explanation: A schema is in 1NF if all attributes are atomic (indivisible) and there are no repeating groups or multi-valued attributes in any column. In our schema, every table (users, platforms, subscription_group, group_members, reviews, plans) strictly contains atomic values. For example, a user has only one phone number per row, and a group has only one plan_description. There are no arrays or comma-separated lists stored in a single column, thus satisfying 1NF.

b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?

A schema is in 2NF if it is in 1NF and there are no partial functional dependencies. A partial functional dependency occurs when a non-prime attribute depends on only a part of a composite primary key. In our relational schema, there cannot be any partial functional dependencies. This is because we specifically designed the database using Single-Attribute Surrogate Primary Keys (e.g., user_id, group_id, review_id, id for group_members) which are all AUTO_INCREMENT integers. Since every primary key consists of only a single column, it is structurally impossible for a non-prime attribute to depend on only a "part" of the primary key. Therefore, every non-prime attribute is fully functionally dependent on the entire primary key, completely satisfying 2NF.

c) Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

Yes, the schema is in Third Normal Form (3NF).
Explanation: A schema is in 3NF if it is in 2NF and there are no transitive dependencies. A transitive dependency occurs when a non-prime attribute depends on another non-prime attribute rather than the primary key (i.e., $X \rightarrow Y \rightarrow Z$). In our schema, there are no transitive dependencies. Every non-key attribute is determined directly by the primary key of its respective table. For example, in the subscription_group table, attributes like group_name, cost_per_member, and validity_start all depend directly on the group_id. While the table contains foreign keys (like owner_id and platform_id), we do not store redundant data (like the

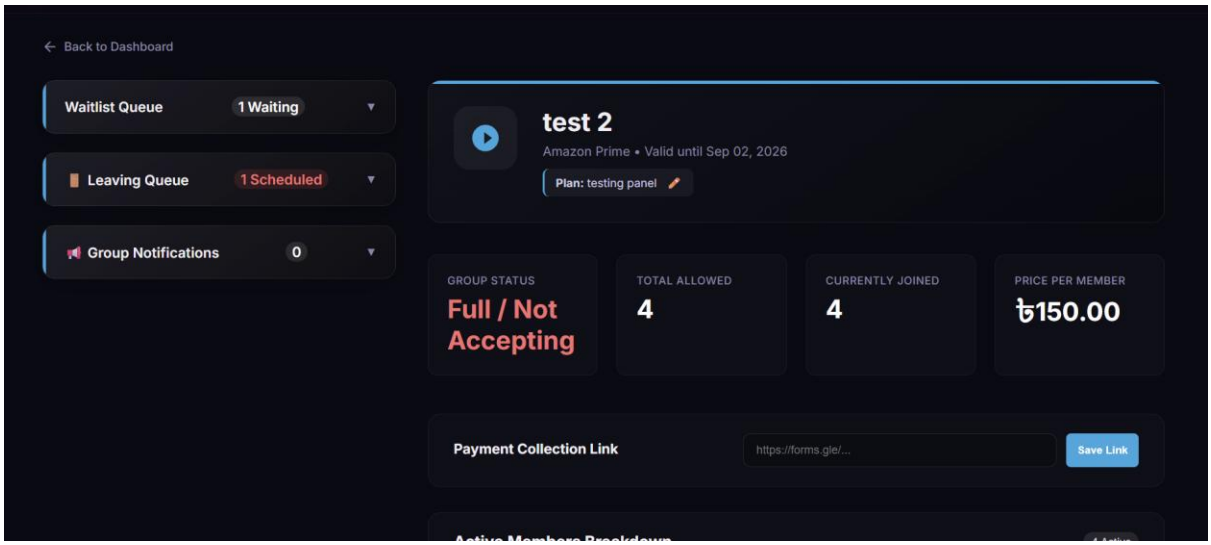
owner's name or the platform's logo) inside the subscription_group table. Instead, we use JOIN operations to retrieve that data, explicitly preventing transitive dependencies and update anomalies. Thus, the database safely satisfies 3NF.

Frontend Development

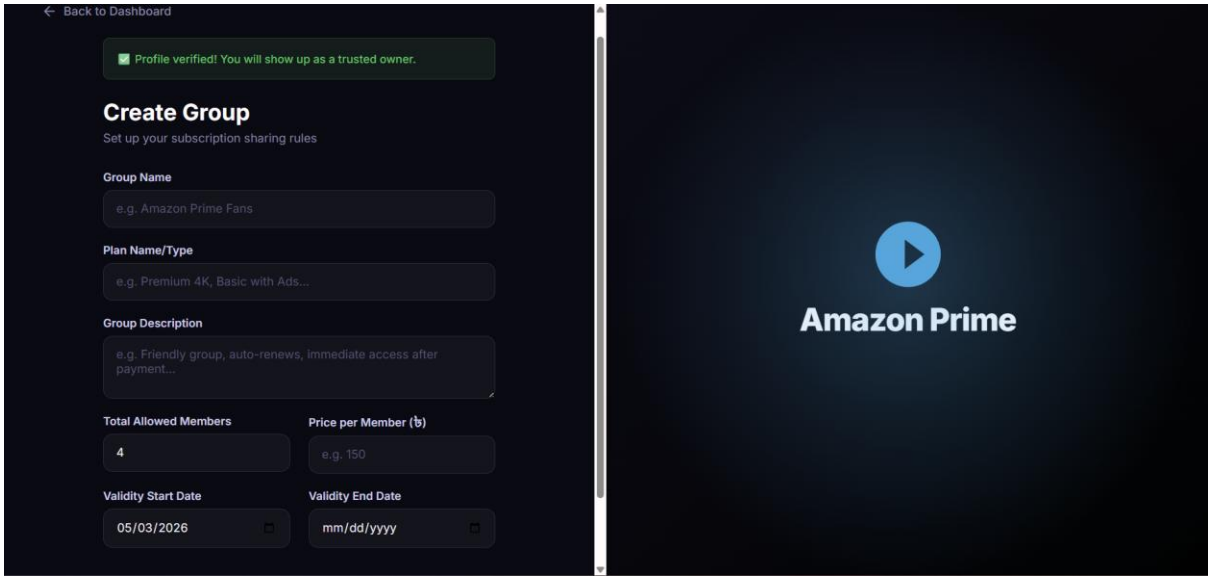
Briefly discuss about Frontend Development and add relevant Screenshots (if required) by mentioning Individual Contributions

Contribution of ID: 24101562, Name: Shadman Shakib

1) owner dashboard from a basic vertical stack into a two-column layout using CSS grid where primary feature is kept at the top and secondary feature is kept on the side panel which has a drop-down option

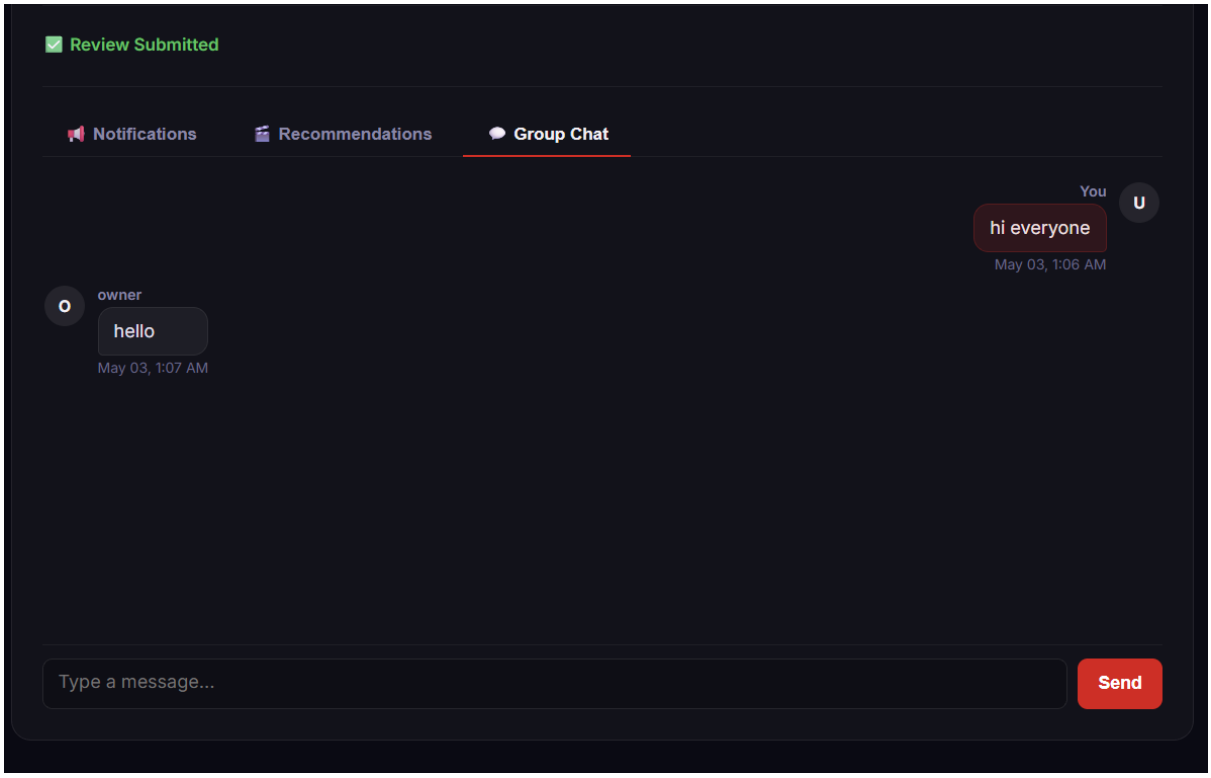


2) In create group section, I have created a dark mode Ui with translucent overlays

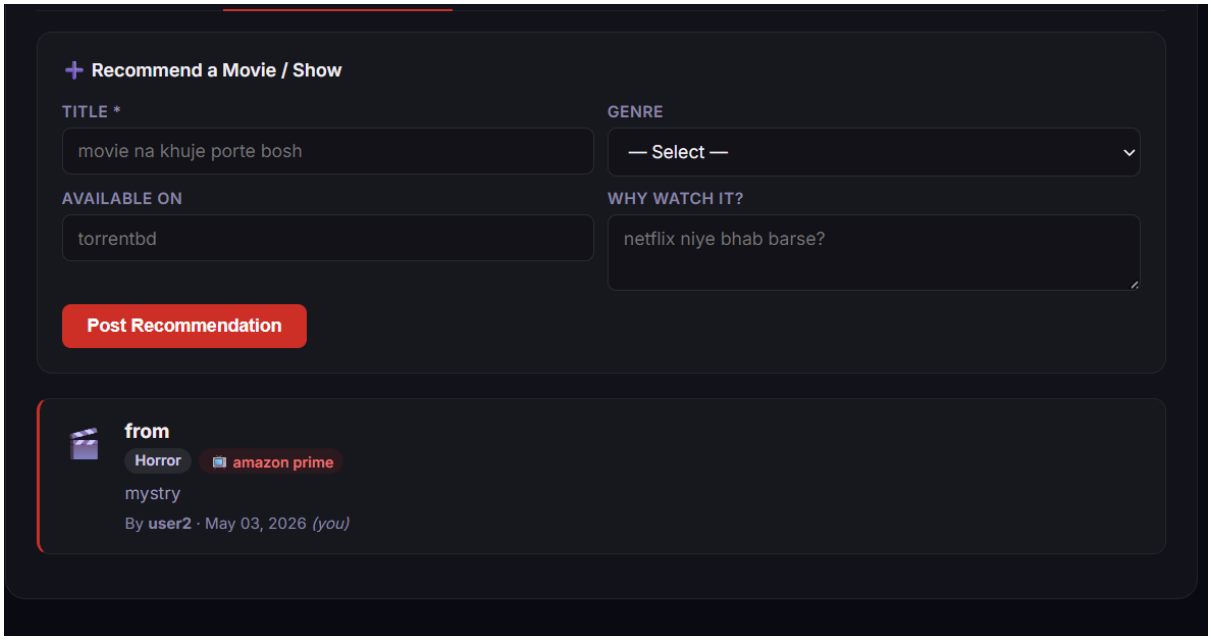


Contribution of ID: 24101175, Name: Tanvir Muhtady

1. I implemented a chatroom for a community messaging among group members which is a responsive, real time conversational interface.

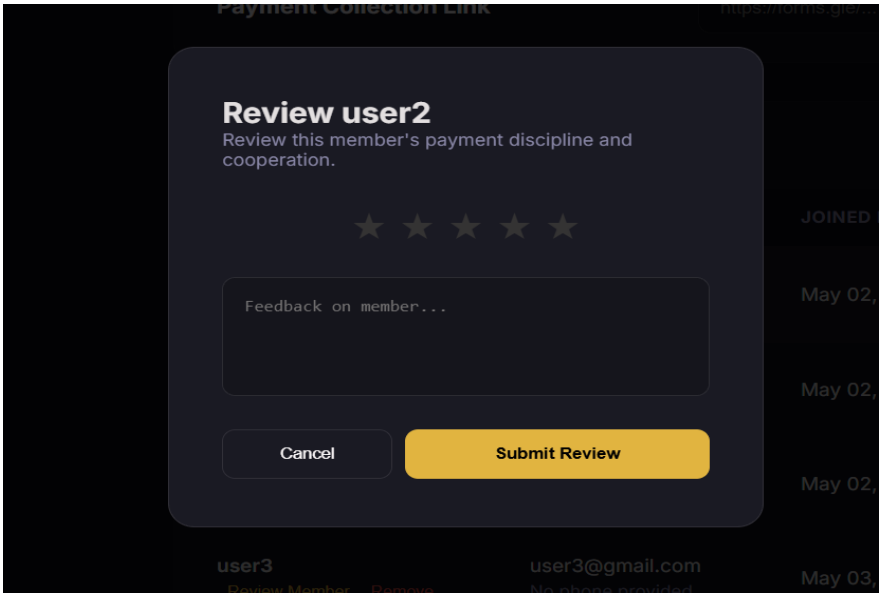


2. Created a frontend layout to display recommended movie among the members

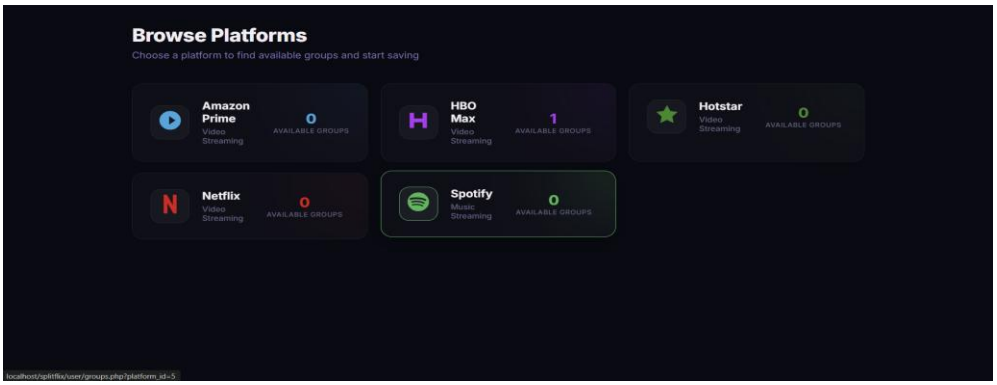


Contribution of ID: 23201017, Name: Tanvir Ahmed

1) I designed the owner member review system frontend



2) I designed the user dashboard panel



Backend Development

Briefly discuss about Backend Development and add relevant Screenshots (if required) by mentioning Individual Contributions

Contribution of ID: 24101562, Name: Shadman Shakib

1) Advanced Membership & Waitlist Architecture: Developed the backend logic to handle complex membership states (Active, Waitlisted, Scheduled Leave). Implemented an automated database trigger/transaction in PHP that automatically deletes a user from *all* other waitlists globally once an owner accepts them into a group, keeping the database clean

```php
// 2. Start Transaction
mysqli_begin_transaction($conn);

try {
    // 2.5 Get user_id for the membership
    $get_user_query = "SELECT user_id FROM group_members WHERE id = ? AND group_id = ?";
    $stmt_user = mysqli_prepare($conn, $get_user_query);
    mysqli_stmt_bind_param($stmt_user, "ii", $membership_id, $group_id);
    mysqli_stmt_execute($stmt_user);
    $res_user = mysqli_stmt_get_result($stmt_user);
    if (!$res_user || mysqli_num_rows($res_user) === 0) {
        throw new Exception("Membership record not found");
    }
    $member_user_id = mysqli_fetch_assoc($res_user)['user_id'];
    mysqli_stmt_close($stmt_user);

    // 3. Update membership status
    $update_member = "UPDATE group_members SET membership_status = 'active' WHERE id = ? AND group_id = ? AND membership_status = 'waitlisted'";
    $stmt1 = mysqli_prepare($conn, $update_member);
    mysqli_stmt_bind_param($stmt1, "ii", $membership_id, $group_id);
    mysqli_stmt_execute($stmt1);

    if (mysqli_stmt_affected_rows($stmt1) === 0) {
        throw new Exception("Membership not found or already active");
    }
    mysqli_stmt_close($stmt1);

    // 4. Update seats remaining
    $update_group = "UPDATE subscription_group SET seats_remaining = seats_remaining - 1 WHERE group_id = ?";
    $stmt2 = mysqli_prepare($conn, $update_group);
    mysqli_stmt_bind_param($stmt2, "i", $group_id);
    mysqli_stmt_execute($stmt2);
    mysqli_stmt_close($stmt2);

    // 5. Delete waitlist requests for this user in all other groups
    $delete_waitlist = "DELETE FROM group_members WHERE user_id = ? AND group_id != ? AND membership_status = 'waitlisted'";
    $stmt3 = mysqli_prepare($conn, $delete_waitlist);
    mysqli_stmt_bind_param($stmt3, "ii", $member_user_id, $group_id);
    mysqli_stmt_execute($stmt3);
    mysqli_stmt_close($stmt3);

    mysqli_commit($conn);
    echo json_encode(['success' => true]);

} catch (Exception $e) {
    mysqli_rollback($conn);
    echo json_encode(['success' => false, 'error' => $e->getMessage()]);
}
?>
```

2) Relational Database Design & Setup: Designed the core SQL schema (group_members, subscription_group, users) with strict foreign key constraints

```sql
CREATE DATABASE IF NOT EXISTS splitflix;
USE splitflix;

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS reviews;
DROP TABLE IF EXISTS group_members;
DROP TABLE IF EXISTS subscription_group;
DROP TABLE IF EXISTS plans;
DROP TABLE IF EXISTS platforms;
DROP TABLE IF EXISTS users;
SET FOREIGN_KEY_CHECKS = 1;
```

```sql
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(20) DEFAULT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('member', 'owner') NOT NULL DEFAULT 'member',
    profile_image VARCHAR(255) DEFAULT NULL,
    id_card_image VARCHAR(255) DEFAULT NULL,
    verification_status ENUM('unverified', 'verified') NOT NULL DEFAULT 'unverified',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```sql
CREATE TABLE platforms (
    platform_id INT AUTO_INCREMENT PRIMARY KEY,
    platform_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL DEFAULT 'Streaming',
    logo_emoji VARCHAR(10) NOT NULL DEFAULT '🎬',
    brand_color VARCHAR(7) NOT NULL DEFAULT '🟥#e50914',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```sql
CREATE TABLE plans (
    plan_id INT AUTO_INCREMENT PRIMARY KEY,
    platform_id INT NOT NULL,
    plan_name VARCHAR(100) NOT NULL,
    max_seats INT NOT NULL DEFAULT 1,
    device_limit INT NOT NULL DEFAULT 1,
    monthly_cost DECIMAL(10,2) NOT NULL,
    billing_type ENUM('monthly', 'yearly') NOT NULL DEFAULT 'monthly',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (platform_id) REFERENCES platforms(platform_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```sql
CREATE TABLE subscription_group (
    group_id INT AUTO_INCREMENT PRIMARY KEY,
    owner_id INT NOT NULL,
    platform_id INT NOT NULL,
    plan_description VARCHAR(255) NOT NULL,
    group_description TEXT DEFAULT NULL,
    group_name VARCHAR(150) NOT NULL,
    max_members INT NOT NULL DEFAULT 4,
    seats_remaining INT NOT NULL DEFAULT 3,
    cost_per_member DECIMAL(10,2) NOT NULL,
    status ENUM('active', 'paused', 'closed') NOT NULL DEFAULT 'active',
    validity_start DATE NOT NULL,
    validity_end DATE DEFAULT NULL,
    contact_info VARCHAR(255) DEFAULT NULL,
    payment_form_link VARCHAR(255) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (owner_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (platform_id) REFERENCES platforms(platform_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

3)Added owner verification

```sql
ALTER TABLE users
ADD COLUMN phone VARCHAR(20) DEFAULT NULL,
ADD COLUMN id_card_image VARCHAR(255) DEFAULT NULL,
ADD COLUMN verification_status ENUM('unverified', 'verified') NOT NULL DEFAULT 'unverified';
```

Contribution of ID: 24101175, Name: Tanvir Muhtady

1) Implemented the backend for chatroom, recommendation and notification broadcast

```sql
CREATE TABLE IF NOT EXISTS group_notifications (
    notification_id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT NOT NULL,
    owner_id INT NOT NULL,
    message TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (group_id) REFERENCES subscription_group(group_id) ON DELETE CASCADE,
    FOREIGN KEY (owner_id) REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE IF NOT EXISTS movie_recommendations (
    recommendation_id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT NOT NULL,
    recommended_by INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    genre VARCHAR(100) DEFAULT NULL,
    description TEXT DEFAULT NULL,
    platform_hint VARCHAR(100) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (group_id) REFERENCES subscription_group(group_id) ON DELETE CASCADE,
    FOREIGN KEY (recommended_by) REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
CREATE TABLE IF NOT EXISTS group_chat (
    message_id INT AUTO_INCREMENT PRIMARY KEY,
    group_id   INT NOT NULL,
    user_id    INT NOT NULL,
    message    TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (group_id) REFERENCES subscription_group(group_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id)  REFERENCES users(user_id) ON DELETE CASCADE,
    INDEX idx_group_created (group_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Contribution of ID: 23201017, Name: Tanvir Ahmed

1) User Dashboard Data Aggregation: Wrote complex SQL JOIN queries to fetch and aggregate a user's entire footprint across the platform. This involved querying multiple tables to simultaneously display groups they actively belong to, groups they are waitlisted for, and groups they are scheduled to leave.

```
$query = "
    (SELECT gm.joined_at, sg.group_id, sg.group_name, sg.plan_description, sg.cost_per_member, sg.validity_end, sg.status as group_status,
            u.name as owner_name, u.email as owner_email, u.phone as owner_phone, u.user_id as owner_id,
            p.platform_name, p.logo_emoji, p.brand_color,
            'member' as my_role,
            gm.payment_status,
            gm.scheduled_leave_date,
            (SELECT review_id FROM reviews WHERE group_id = sg.group_id AND reviewer_id = ? AND reviewer_role = 'member') as my_review_id
    FROM group_members gm
    JOIN subscription_group sg ON gm.group_id = sg.group_id
    JOIN users u ON sg.owner_id = u.user_id
    JOIN platforms p ON sg.platform_id = p.platform_id
    WHERE gm.user_id = ? AND gm.membership_status = 'active')

    UNION ALL

    (SELECT sg.created_at as joined_at, sg.group_id, sg.group_name, sg.plan_description, sg.cost_per_member, sg.validity_end, sg.status as group_status,
            'Me' as owner_name, 'Me' as owner_email, 'Me' as owner_phone, sg.owner_id,
            p.platform_name, p.logo_emoji, p.brand_color,
            'owner' as my_role,
            NULL as payment_status,
            NULL as scheduled_leave_date,
            NULL as my_review_id
    FROM subscription_group sg
    JOIN platforms p ON sg.platform_id = p.platform_id
    WHERE sg.owner_id = ? AND sg.status = 'active')

    ORDER BY joined_at DESC
";
```

2) Platform Browsing & Filtering Logic: Developed the backend logic to query the database for active groups with seats_remaining > 0, allowing users to dynamically browse available subscriptions by platform.

```
$result = mysqli_query($conn, $query);
if ($result) {
    while ($row = mysqli_fetch_assoc($result)) {
        $platforms[] = $row;
    }
}

// Count available groups per platform
$group_counts = [];
$query = "SELECT p.platform_id, COUNT(sg.group_id) as group_count
          FROM platforms p
          LEFT JOIN subscription_group sg ON p.platform_id = sg.platform_id AND sg.status = 'active' AND sg.seats_remaining > 0
          GROUP BY p.platform_id";
$result = mysqli_query($conn, $query);
if ($result) {
    while ($row = mysqli_fetch_assoc($result)) {
        $group_counts[$row['platform_id']] = $row['group_count'];
    }
}
```

3) Bi-directional Review & Rating System: Designed the backend logic for submit_review.php which tracks the reviewer_role (owner vs. user). Implemented complex SQL aggregation functions (AVG ()) to dynamically calculate and return a user's overall trust rating based on historical reviews, while preventing duplicate review submissions.

```php
$dup_check = "SELECT review_id FROM reviews WHERE group_id = ? AND reviewer_id = ? AND reviewer_role = ?";
$stmt = mysqli_prepare($conn, $dup_check);
mysqli_stmt_bind_param($stmt, "iis", $group_id, $reviewer_id, $role);
mysqli_stmt_execute($stmt);
if (mysqli_num_rows(mysqli_stmt_get_result($stmt)) > 0) {
    header("Location: my_groups.php?error=Already reviewed");
    exit();
}
mysqli_stmt_close($stmt);

$insert_query = "INSERT INTO reviews (group_id, reviewer_id, reviewee_id, reviewer_role, rating, comment) VALUES (?, ?, ?, ?, ?,
$stmt = mysqli_prepare($conn, $insert_query);
mysqli_stmt_bind_param($stmt, "iiisis", $group_id, $reviewer_id, $reviewee_id, $role, $rating, $comment);
```

Source Code Repository

Upload source code to GitHub or Google Drive and share a publicly accessible link in this section of the report.

https://github.com/Sakibenth/Splitflix

Conclusion

Splitflix is a comprehensive, web-based subscription-sharing platform designed to elivate the financial burden of premium digital services like Netflix, Amazon Prime, and Spotify. The platform connects users who have unused screens on their subscription plans with individuals looking for affordable access to premium content.

Built using a robust tech stack of PHP, MySQL and CSS Grid, Splitflix operates on a secure, role-based architecture. Users can either act as **Group Owners**, creating and managing subscription groups, or as **Members**, requesting to join available slots. The platform features dynamic capacity tracking, a sophisticated automated waitlist system, strict time-based billing security, and a bi-directional trust and review system. Furthermore, Splitflix enhances user engagement through real-time interactive features, including a built-in group chatroom, an algorithmic recommendation engine, and an owner broadcasting panel. Ultimately, Splitflix provides a safe, organized, and highly responsive environment for managing shared digital expenses.

References

1)MDN Docs
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides
2)w3 Schools
https://www.w3schools.com/php/
https://www.w3schools.com/MYSQL/default.asp
3)php tutorial
https://www.phptutorial.net/
4)Learn php the right way
https://phptherightway.com/
5)Learn Mysql
https://www.mysqltutorial.org/